ScrollGuard 에이전트 벤치마크 기술 보고서

Phase 1 벤치마크 제작 · 생성형인공지능특론 · 2026-10-28

1. 요약

ScrollGuard(의도 인식형 스크롤 방지 에이전트)가 '지금 개입해야 하는가'를 사용 맥락까지 읽고 판단하는지 평가하는 벤치마크를 구축했다. 사례 45개(반례·경계 사례 포함), 입력/정답 분리, 지표·기준선 채점 파이프라인으로 구성된다.

현재 상태: 기준선 5종의 채점까지 완료했다. 실제 LLM 에이전트 실행과 라벨의 독립 검증은 아직 수행하지 않았다(이 두 가지가 남은 핵심 작업).

2. 요구사항 달성도

과제 요구사항: (a) 에이전트가 수행할 태스크 정의, (b) 테스트 케이스/평가 데이터 준비, (c) 성공 기준과 지표 수립, 그리고 벤치마크로 에이전트의 목표 달성도를 평가할 수 있어야 함(Evaluating LLM 강의 기준).

요구사항	달성 수준	근거	남은 것
(a) 태스크 정의	충족	AGENT_SPEC.md에 입력 스키마, 출력 JSON, 행동 3개, 라벨 경계 규칙 명시	없음
(b) 테스트 케이스	대부분 충족 (자체 평가 약 75%)	45개(18/17/10), 반례 9개, 경계 8개, 숫자 4개는 실제 데이터 행, 입력/정답 분리, 누출 점검 통과	라벨 독립 검증, 합성 신호의 현실성, 표본 크기
(c) 성공 기준·지표	충족	탐지율, 불필요 개입률, 맥락확인율, 비용 점수 등 구현, 기준선 5종 채점 완료	비용 가중치 민감도 분석
에이전트 평가 가능성 (Evaluating LLM 관점)	부분 (자체 평가 약 40%)	파이프라인 end-to-end 동작 확인(mock)	실제 에이전트 실행, 3회 반복 일관성, 오류 분석

※ 퍼센트는 요구사항 대비 작업 완성도에 대한 자체 판단이며 측정값이 아니다.

3. 벤치마크 설계

3.1 에이전트 태스크

사용 상황 한 건을 입력받아 세 행동 중 하나를 고른다.

- 입력(12열): 시각, 진입 계기, 앱 유형, 사용자 선언 목표, 최근 맥락 텍스트, 최근 60분 사용 분·세션 수·중앙 세션 길이·집중도, 스와이프 간격 변동계수, 콘텐츠당 평균 체류
- 출력: JSON 한 줄 {"action": "...", "reason": "..."}
- 허용 행동: no_intervention / gentle_prompt / ask_context (그 외는 invalid)

3.2 라벨 정의

행동 (의도)	기준
no_intervention (intentional)	선언 목표와 행동 일치, 목적 행동(검색·저장·작성·앱 전환) 관찰, 짧은 유틸리티 사용, 사용자 설정 시간 내 사용. 길어도 개입하지 않음
gentle_prompt (unintended)	자동조종 신호가 2개 이상 독립적으로 있고 반대 증거가 없음 (목적 행동 부재, 균일한 스와이프·짧은 체류, 알림 진입 후 피드 이탈, 선언 목표·타이머·취침 목표 이탈, 일정 충돌 등)
ask_context (unclear)	목적 신호와 이탈 신호가 충돌하거나, 핵심 정보가 없는데 사용이 눈에 띔. 틀린 개입 비용이 질문 비용보다 큼

경계 규칙: 사용자의 명시적 선언은 존중한다. 시간 초과 단독은 근거가 부족하다. 1~3분 사용에 정보가 없으면 질문하지 않는다.

3.3 사례 구성

총 45개: no_intervention 18, gentle_prompt 17, ask_context 10. 유형은 standard 28, adversarial 9(시간·세션·집중도만 보면 틀리게 설계), boundary 8. 숫자 4개는 실제 데이터이고 시각·계기·앱 유형·스와이프·체류·맥락 텍스트는 합성이다.

3.4 지표와 기준선

- 정확도(+부트스트랩 95% CI), macro-F1, 혼동행렬
- 탐지율: gentle_prompt 사례를 맞춘 비율
- 불필요 개입률: no_intervention 사례에 gentle_prompt를 낸 비율 (제품 철학상 핵심)
- 불필요 질문률 / 맥락확인율: no_intervention에 ask_context를 낸 비율 / ask_context 사례를 맞춘 비율

- 비용 점수 = 1 − 평균 비용 (불필요 개입 1.0 > 놓침 0.8 > 오답 질문 0.4~0.6, 설계 가정)
- hard(adversarial+boundary) 정확도, invalid 출력률, 같은 사례 3회 실행 일관성
- 기준선: B0 항상 no_intervention / 항상 gentle_prompt / 항상 ask_context, B1 사용 30분 이상이면 개입, B2 텍스트 없이 숫자·시각·진입 계기만 쓰는 규칙

4. (2) 파일별 역할

파일	역할	입력 → 출력
micro_datasetA.csv	공개 원본 데이터 (5,777행 × 17열)	build_benchmark.py의 입력
inspect_dataset.py, prepare_benchmark_data.py, sample_cases.py	v1 데이터 탐색 (유지)	micro_datasetA.csv → 콘솔, sample_cases.csv
build_benchmark.py	45개 사례 정의, 구간별 실제 행 추출, 파일 생성. v1의 create_benchmark_cases.py와 split_benchmark.py를 대체	micro_datasetA.csv → inputs, labels, meta, annotation_sheet
benchmark_inputs.csv	에이전트에 제공하는 입력 (정답 없음)	run_agent.py, check_leakage.py, evaluate.py가 읽음
benchmark_labels.csv	정답 (user_intent, intervention_needed, expected_action). 에이전트에 비공개	evaluate.py, agreement.py가 읽음
benchmark_meta.csv	사례 유형, 라벨 근거, 원본 행 출처. 비공개	evaluate.py가 읽음
annotation_sheet.csv	입력 + 빈 annotator_action 열. 독립 라벨링용	라벨러가 채워 agreement.py에 입력
check_leakage.py	입력 텍스트에 정답을 직접 서술하는 표현이 있는지 점검	benchmark_inputs.csv → 콘솔
run_agent.py	입력만 LLM에 전달하고 JSON을 파싱해 예측 저장. --mock은 랜덤 (파이프라인 점검용)	benchmark_inputs.csv → predictions_*.csv
evaluate.py	기준선 5종과 예측을 채점. 지표, CI, 혼동행렬 출력	inputs, labels, meta, predictions → 콘솔, results_summary.csv

파일	역할	입력 → 출력
agreement.py	라벨러 간·라벨러-설계자 간 Cohen's kappa	labels, 라벨러 CSV → 콘솔
AGENT_SPEC.md	태스크·라벨·지표 명세 문서	제출 문서의 근거
results_summary.csv	마지막 evaluate.py 실행의 지표표	evaluate.py가 생성

5. 실행 흐름과 호출 관계

```
micro_datasetA.csv
      |
      v
build_benchmark.py --> benchmark_inputs.csv --> check_leakage.py (누출 점검)
      |                        |
      |                        v
      |                run_agent.py --> predictions_*.csv --+
      +--> benchmark_labels.csv ----------------------------+
      +--> benchmark_meta.csv -------------------------------+--> evaluate.py
--> results_summary.csv
      +--> annotation_sheet.csv --> (독립 라벨러 작성) --> agreement.py
```

함수 수준의 호출 관계는 다음과 같다. 세 스크립트는 서로 import하지 않고 CSV 파일로만 연결된다.

- build_benchmark.py: CASES(45개 정의) → TIERS 조건으로 data.query() → 무작위 행 선택(seed 42) → 4개 CSV 저장
- run_agent.py: main() → build_user_message(행) → call_llm() → parse() → CSV 저장. 정답 파일은 읽지 않는다.
- evaluate.py: main() → BASELINES 5개 함수 실행 + 예측 파일 로드 → evaluate() → score_run() → macro_f1() → 표 출력, results_summary.csv 저장

6. 실행 방법과 터미널 결과

6.1 준비

모든 파일을 한 폴더에 둔다(micro_datasetA.csv 포함). PowerShell에서 그 폴더로 이동하고 패키지를 설치한다. anthropic은 실제 에이전트 실행 때만 필요하다.

```
cd (파일을 둔 폴더 경로)
pip install pandas numpy anthropic
```

한글이 깨지면 chcp 65001을 먼저 실행한다.

6.2 벤치마크 생성

```
python build_benchmark.py
```

터미널 결과:

```
생성 완료: 45 사례
expected_action
no_intervention    18
gentle_prompt      17
ask_context        10
case_type
standard       28
adversarial     9
boundary        8
```

생성되는 파일: benchmark_inputs.csv, benchmark_labels.csv, benchmark_meta.csv, annotation_sheet.csv

6.3 누출 점검

```
python check_leakage.py
```

```
누출 의심 표현: 없음
```

6.4 기준선 채점

```
python evaluate.py --confusion
```

터미널 결과(표 일부 열 생략 없이 출력된 그대로, 이어서 혼동행렬 중 B1·B2):

```
                         accuracy      acc_ci95 macro_f1 detect_rate
false_intervention question_burden ask_recall hard_acc cost_score
invalid_rate consistency  n_runs
B0_always_no_intervention     0.40  [0.27, 0.53]     0.19        0.00
0.00            0.00       0.00     0.47       0.56         0.00         nan
1
B0_always_gentle_prompt       0.38  [0.24, 0.51]     0.18        1.00
1.00            0.00       0.00     0.24       0.47         0.00         nan
1
B0_always_ask_context         0.22  [0.11, 0.36]     0.12        0.00
0.00            1.00       1.00     0.29       0.65         0.00         nan
```

```
1
B1_time_threshold_30min       0.47  [0.33, 0.62]     0.35        0.53
0.33            0.00       0.00     0.24       0.59         0.00         nan
1
B2_feature_rule_no_text       0.58  [0.42, 0.73]     0.43        0.47
0.00            0.00       0.00     0.71       0.71         0.00         nan
1

[B1_time_threshold_30min] 행=정답, 열=예측
col_0            no_intervention  gentle_prompt
expected_action
no_intervention               12              6
gentle_prompt                  8              9
ask_context                    5              5

[B2_feature_rule_no_text] 행=정답, 열=예측
col_0            no_intervention  gentle_prompt
expected_action
no_intervention               18              0
gentle_prompt                  9              8
ask_context                   10              0
```

결과 파일 results_summary.csv도 함께 저장된다.

6.5 에이전트 실행

파이프라인 점검(랜덤 mock, 성능 의미 없음):

```
python run_agent.py --mock --runs 3 --out predictions_mock.csv
```

```
run 0 완료
run 1 완료
run 2 완료
저장: predictions_mock.csv
```

실제 에이전트 실행(API 키 필요, 모델명은 사용하는 모델로 교체):

```
$env:ANTHROPIC_API_KEY = "본인 API 키"
python run_agent.py --model claude-sonnet-5-5 --runs 3 --out
predictions_agent.csv
```

출력 형식은 mock과 같고 마지막 줄이 저장: predictions_agent.csv가 된다. 다른 LLM(OpenAI 등)을 쓰려면 run_agent.py의 call_llm() 함수 하나만 해당 API 호출로 바꾼다. 45사례 × 3회 = 호출 135건이 발생한다.

6.6 에이전트 채점

```
python evaluate.py --pred predictions_agent.csv --confusion
```

기준선 5행 아래에 predictions_agent 행이 추가된다. mock 파일로 실행했을 때의 해당 행은 다음과 같았다(랜덤이므로 기준선보다 낮은 것이 정상).

```
predictions_mock              0.29  [0.21, 0.37]     0.27        0.33
0.26            0.41       0.13     0.27       0.56         0.00        0.11
3
```

여러 에이전트를 비교하려면 --pred 뒤에 파일을 여러 개 적는다.

6.7 라벨 독립 검증

annotation_sheet.csv를 2명에게 주고 annotator_action 열(no_intervention / gentle_prompt / ask_context 중 하나)을 채우게 한 뒤 실행한다.

```
python agreement.py annot_a.csv annot_b.csv
```

출력 형식(아래는 정답을 그대로 복사한 파일로 돌린 동작 확인 결과이며 실제 라벨러 결과는 이보다 낮게 나온다):

```
annot_a.csv vs 설계자 라벨: 일치 1.00, kappa 1.00
해석 참고: kappa 0.6 이상이면 '상당한 일치', 0.4 미만이면 라벨 기준 재정의 필요
```

라벨러가 둘이면 annot_a.csv vs annot_b.csv 줄도 함께 출력된다.

7. 현재 결과의 해석

- 임계값 규칙(B1)은 의도적 사용의 33%에 불필요하게 개입한다. 시간만 보는 방식(Context-Blind)의 한계가 수치로 나타난다.
- 기준선은 모두 ask_context를 낼 수 없어 맥락확인율이 0이다(항상 질문하는 기준선 제외). 맥락 판단은 에이전트의 고유 가치가 될 수 있는 영역이다.
- 에이전트 목표: B2(정확도 0.58, 비용 점수 0.71)를 넘으면서 불필요 개입률을 낮게 유지하고, 맥락확인율을 0보다 크게 만드는 것.
- 주의: B2는 사례를 본 뒤 설계한 규칙이라 낙관적인 상한선이다. 45개 표본에서는 CI가 넓어(B2: 0.42~0.73) 작은 차이를 해석하면 안 된다.

8. 한계와 향후 과제

1. 라벨 타당성: 라벨은 설계자 정의 기대값이다. 독립 라벨러 2명의 kappa를 보고해야 타당성을 주장할 수 있다.

2. 합성 신호: 시각·계기·스와이프·체류·맥락 텍스트는 실제 관측이 아니며, 사용자의 실제 개입 선호를 조사한 데이터가 아니다.
3. 표본 크기: 45개는 여전히 작다. 검증 후 100개 안팎으로 확장하는 것이 바람직하다.
4. 설계 가정: 비용 가중치와 '신호 2개 이상' 기준은 가정이다. 가중치를 바꿔 순위가 유지되는지 민감도 분석이 필요하다.
5. 누출 점검의 범위: check_leakage.py는 키워드 수준이다. 문체나 문장 길이 같은 암묵적 단서는 점검하지 않는다.
6. Phase 2 운영: 다른 팀 에이전트를 평가하려면 benchmark_labels.csv와 benchmark_meta.csv를 비공개로 두고, 예측 파일 형식(case_id, run, action, reason)을 공지하며, evaluate.py의 --pred로 리더보드를 만들 수 있다.
7. 에이전트 실행 후 case_type별 오류 분석 1쪽을 추가한다.